




# RippleGUItester: Change-Aware Exploratory Testing

YANQI SU, Technical University of Munich, Germany
MICHAEL PRADEL, CISPA Helmholtz Center for Information Security, Germany
CHUNYANG CHEN, Technical University of Munich; Fortiss; Munich Data Science Institute, Germany

Software systems evolve continuously through frequent code changes, yet such changes often introduce unintended bugs despite extensive testing and code review. Existing testing approaches are largely constrained to predefined execution paths or rely on unguided exploration, leaving many change-induced issues undetected. To address this challenge, we present *RippleGUItester*, a change-driven testing system that treats a code change as the epicenter of a ripple effect and explores its broader, user-visible impacts via the GUI. Given a code change, RippleGUItester performs LLM-based change-impact analysis to generate and enrich realistic test scenarios, executes these scenarios on both pre-change and post-change versions of the system, and applies differential analysis to identify behavioral differences. Crucially, RippleGUItester employs multimodal bug detection, comparing visual GUI changes and interpreting them in the context of natural-language change intents to distinguish unintended bugs from intended behavioral updates. We evaluate our approach on hundreds of real-world code changes across four widely used software systems: Firefox, Zettlr, JabRef, and Godot. Our results show that the proposed approach uncovers bugs introduced by code changes that were missed by existing test suites, CI pipelines, and code review. In total, we identify and report 26 previously unknown bugs that still exist in the latest versions of the evaluated systems. 18 bugs have been fixed, 3 have been confirmed, and 5 were marked as intended. We envision RippleGUItester being applied before or shortly after a code change is merged, enabling earlier detection of regressions.

CCS Concepts: • **Software and its engineering → Software testing and debugging**.

Additional Key Words and Phrases: Change-Aware Testing, Exploratory Testing, Large Language Models

**ACM Reference Format:**
Yanqi Su, Michael Pradel, and Chunyang Chen. 2026. RippleGUItester: Change-Aware Exploratory Testing. *Proc. ACM Softw. Eng.* 3, ISSTA, Article ISSTA149 (October 2026), 22 pages. https://doi.org/10.1145/3832240

## 1 Introduction

Modern software systems evolve continuously through frequent code changes. While such changes are essential for adding features, fixing defects, and improving maintainability, they also risk introducing new bugs. Despite extensive testing and code review, a non-negligible fraction of code changes still introduce new bugs [17, 20, 24, 29, 37, 40], negatively affecting user experience and incurring substantial debugging and maintenance costs.

To better understand this phenomenon, we analyze code changes and the bugs they introduce. Even with rigorous testing, 12.2% of analyzed code changes introduced new bugs, highlighting the prevalence of this problem in real-world software development. Fig 1 shows a representative example from the Firefox browser. When fixing Issue 1858633 (*Passwords page opens with incorrect URL fragment*), which is triggered by opening the Passwords page from the App Menu while a site is open, the code change unintentionally introduced another bug (Issue 1937085). This new bug is

Authors' Contact Information: Yanqi Su, yanqi.su@tum.de, Technical University of Munich, Germany; Michael Pradel, michael@binaervarianz.de, CISPA Helmholtz Center for Information Security, Germany; Chunyang Chen, chunyang.chen@tum.de, Technical University of Munich; Fortiss; Munich Data Science Institute, Germany.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2994-970X/2026/10-ARTISSTA149
https://doi.org/10.1145/3832240

triggered when the Passwords page is accessed from a pop-up window, causing it to open in the pop-up rather than the main browser window. This example highlights several key challenges in detecting bugs introduced by code changes: (i) diverse triggering event sequences, such as accessing the same page through a pop-up window instead of the App Menu; (ii) difficulty in constructing appropriate test data, for instance requiring a website that can trigger a login pop-up window (e.g., https://x.com/); and (iii) hard-to-predict test oracles for GUI behavior, such as whether the Passwords page opens in the main window.

Bugs may also arise from cross-scenario side effects, where fixing one usage scenario inadvertently disrupts another seemingly unrelated one. For example, fixing a missing scrollbar on the Settings page (Issue 1792881) unintentionally caused the search box to shift when no search results were returned (Issue 1793730). Although both issues occur within the Settings page, they are triggered by different usage scenarios, scrolling versus performing a search, making such side effects difficult to anticipate during testing. Thus, many bugs escape existing testing procedures, such as regression testing and automated CI pipelines, which are largely constrained to predefined execution paths and expected behaviors. Exploratory testing [9, 18, 38], while effective at uncovering unexpected issues, is not driven by code changes and provides little systematic guidance on what to explore after a change. This gap motivates the need for a testing approach that is explicitly driven by code changes yet capable of exploring their broader effects.

This work presents *RippleGUItester*, a change-driven testing system that aims to uncover user-visible issues induced by a code change by systematically exploring the ripple effects of that change via the GUI. Rather than focusing solely on the intended modification, RippleGUItester enables the discovery of indirect and cross-scenario effects that are difficult to anticipate upfront and are often missed by existing testing approaches. RippleGUItester consists of three components: Test Scenario Generator, Test Scenario Executor, and Bug Detector. Starting from a given code change, the Generator performs LLM-based change-impact analysis to identify potentially affected behaviors and generate initial test scenarios. These scenarios are enriched with diverse triggering event sequences derived from scenario knowledge mined from historical issues and pull requests. To make scenarios executable, concrete test data is instantiated, producing realistic and fully executable test scenarios. The Executor then runs each scenario on both the pre-change and post-change builds of the software under test. Finally, the Detector adopts a differential analysis strategy. It detects and annotates visual differences between GUI screenshots from the two builds and analyzes them in the context of the executed scenario and the change intent to determine whether an observed difference corresponds to an intended behavior change or an unintended bug.

Our evaluation applies RippleGUItester to hundreds of real-world code changes from four widely used and complex software systems: Firefox, Zettlr, JabRef, and Godot. The results show that RippleGUItester is able to detect bugs introduced by code changes that remain unnoticed by existing test suites, CI tools, and code review. Overall, we uncovered and reported 26 previously unknown bugs that still exist in the latest versions of the evaluated systems. 18 bugs have been fixed, 3 have been confirmed, and 5 were marked as intended. These results indicate that RippleGUItester is effective in uncovering unintended, user-visible behavioral changes caused by code change. While RippleGUItester incurs relatively high overhead (54.8 minutes and $6.00 per pull request on average), it demonstrates the ability to uncover real-world GUI regressions that are difficult to detect both manually and automatically.

In summary, this paper makes the following contributions:

- We observe and analyze code changes and the bugs they introduce in the real-world software system, quantifying their prevalence and highlighting the practical challenges that make such bugs difficult to detect.

- We introduce RippleGUItester, the first change-aware GUI testing system that uncovers user-visible issues induced by code changes by systematically exploring their ripple effects across usage scenarios.
- We evaluate RippleGUItester on hundreds of real-world code changes from four widely used software systems, demonstrating its effectiveness by uncovering 26 previously unknown bugs in the latest versions of the evaluated systems: 18 bugs have been fixed, 3 have been confirmed, and 5 were marked as intended.
- We make our dataset and implementation publicly available to foster future research (see Section 7 for details).

## 2 Motivation

Code changes, whether for implementing tasks, requesting features, fixing bugs or refactoring, can inadvertently introduce new bugs. To better understand this phenomenon, we examine how often code changes result in new bugs and identify challenges that make such bugs difficult to anticipate and detect in practice. Throughout this paper, we use the term *Issue* to denote any task, feature, bug, or refactoring, and the term *Pull Request (PR)* to denote the code changes made to fix an issue.

### 2.1 Prevalence of Bug-Inducing Code Changes

To investigate the prevalence of this phenomenon, we analyzed 97,347 PRs merged to the Firefox between May 1, 2019, and January 1, 2025. We selected Firefox as the study subject due to its comprehensive bug management system, Bugzilla. This system provides traceability between PRs and both the issues they resolve and the unintended bugs they introduce. Despite rigorous quality assurance procedures such as linting, unit testing, regression testing, and code review prior to integration into the source code repository, 11,910 PRs still introduced new bugs, accounting for 12.2% of all changes. This percentage represents a conservative estimate, as some bugs are never detected and others may be detected but not linked to the bug-introducing PR. This indicates that the phenomenon is widespread in real-world practice. The bugs are typically discovered by users, testers, or developers in the subsequent versions. Evidently, this not only impacts user experience but also incurs significant testing, debugging and fixing overhead.

Despite rigorous testing, 11,910 PRs (12.2% of all changes) still introduced new bugs, underscoring the widespread and significant nature of this phenomenon.

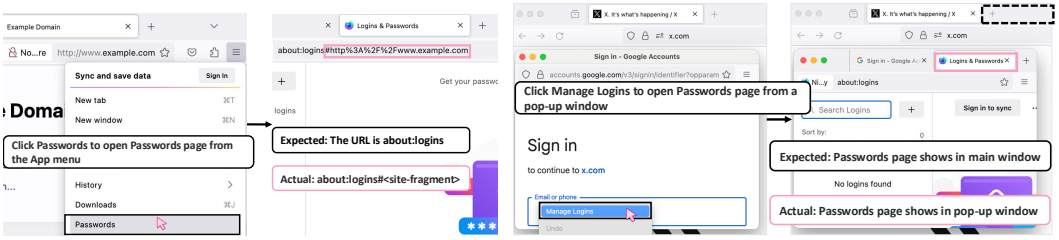
### 2.2 Challenges of Change-Induced Bug Detection

Our analysis shows that these bugs are not simple regressions and therefore cannot be reliably detected by rerunning existing regression tests. Instead, they often involve complex triggering steps and hard-to-predict test oracles. We illustrate these challenges with concrete examples.

*2.2.1 Challenging Trigger Steps.* Trigger steps consist of two key parts: event sequences and test data. *Event sequences* are the sequences of user actions or system events that drive the software into specific states. *Test data* are sets of inputs or information used to verify the correctness, performance, and reliability of software systems. We find that the main challenge in triggering introduced bugs is the diversity of event sequences and test data involved in complex software systems.

**Diverse Event Sequences.** As shown in Fig 1a, Issue 1858633 (*Passwords page opens with incorrect URL fragment*) is triggered when the Passwords page is opened from the App Menu while a website is open. The PR that fixed this issue introduced another bug (Issue 1937085: *about:logins opens in Sign-In popup window instead of main Firefox window*). As shown in Fig 1b, this bug is triggered when the Passwords page is opened from a pop-up window. Specifically, the event

sequence involves opening a website with a pop-up login window (e.g., https://x.com/), clicking the "Sign in" button to activate the pop-up, right-clicking the input field, and selecting "Manage Logins". Thus, detecting bugs during the code change stage requires considering diverse event sequences.




(a) Issue1858633: Open with site fragment in URL (b) Issue1937085: Open in pop-up window

Fig. 1. Fixing Issue1858633 introduces Issue1937085. Only the final event is shown.

**Diverse Test Data.** Issue 1750072 introduces a built-in password reveal/conceal control for password inputs. Testing this enhancement requires exercising the same event sequences, i.e., opening a UI containing password fields and observing the reveal/conceal control. These different UIs containing password fields constitute distinct test data. However, due to the large number of such UIs, exhaustive testing is impractical. As a result, testing overlooked the Thunderbird mail account setup page (Issue 1913889: *Two buttons to toggle password visibility*), and the Save Password doorhanger (Issue 1936548: *Redundant Show password option*). Thus, generating representative and diverse test data is essential for adequately validating code changes.

**Cross-Scenario Side Effects.** PRs that resolve a specific issue may also affect other usage scenarios that appear unrelated at first glance. For example, fixing Issue 1792881 (a missing vertical scrollbar on the Settings page) unintentionally caused Issue 1793730, where the search box and page content shift when no search results are returned. Although both issues occur within the same Settings interface, they correspond to different user interaction scenarios: scrolling the settings page versus performing a search that yields no results. When testing the fix for scrollbar issue, it is difficult to anticipate that the search-related scenario should also be tested. This example highlights the challenge of uncovering hidden dependencies across seemingly unrelated usage scenarios.

*2.2.2 Hard-to-Predict Test Oracles.* To catch introduced bugs, it is not enough to only trigger them. Corresponding test oracles are also required to detect their manifestations. However, as the examples above illustrate, the test oracles are highly irregular and hard-to-predict. For instance, fixing the incorrect Passwords URL led to the Passwords page opening in a pop-up window; enabling the built-in password reveal/conceal control resulted in redundant "Show password" options, and resolving the missing vertical scrollbar caused the search bar to shift unexpectedly. These cases demonstrate that rule-based test oracles are infeasible. Being able to capture such irregular and hard-to-predict oracles remains another major challenge.

Detecting bugs is inherently challenging because introduced bugs often require diverse event sequences, varied test data, and interactions across multiple usage scenarios to be triggered. Moreover, their manifestations depend on hard-to-predict test oracles. These factors together explain why many bugs escape existing testing procedures.

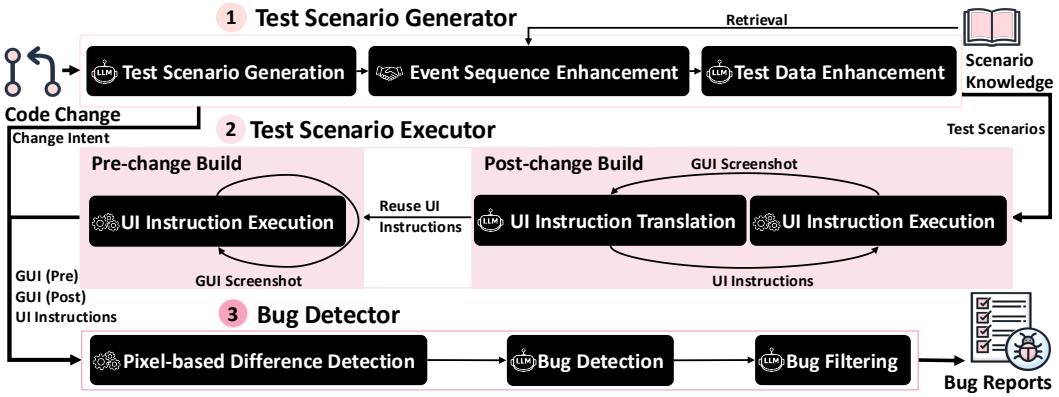



Fig. 2. Approach Overview. Icons denote types: LLM-based (robot), algorithmic (gear), and hybrid (handshake).

## 3 Approach

To capture bugs introduced by code changes, we present *RippleGUItester*, a novel testing technique that integrates the natural language understanding, code comprehension, and vision capabilities of multimodal LLMs with application-specific knowledge. As shown in Fig 2, the system comprises three components: Test Scenario Generator, Test Scenario Executor, and Bug Detector (hereafter abbreviated as *Generator*, *Executor*, and *Detector*). The *Generator* analyzes a given code change to identify potentially affected usage scenarios and generate executable test scenarios (Section 3.1). The *Executor* runs these scenarios on both the pre-change and post-change builds of the SUT (Section 3.2). The *Detector* compares execution results across the two builds to determine whether observed differences correspond to intended behavioral changes or unintended bugs (Section 3.3).

### 3.1 Test Scenario Generator

The Generator analyzes the potential impacts of code changes and generates corresponding test scenarios by incorporating application-specific knowledge. As discussed in Section 2.2, change-induced bugs often involve challenging event sequences, diverse test data, and cross-scenario side effects. To address these challenges, the Generator consists of three subcomponents: *Test Scenario Generation*, *Event Sequence Enhancement*, and *Test Data Enhancement*. Given a PR, *Test Scenario Generation* performs LLM-based change-impact analysis to identify potentially affected usage scenarios and produces initial test scenarios. *Event Sequence Enhancement* then retrieves additional application-specific knowledge via targeted queries and uses it to enrich the event sequences of these scenarios. Finally, *Test Data Enhancement* identifies required test data and constraints, and generates concrete data instances, producing executable test scenarios with realistic test data.

*3.1.1 Input Preparation.* To generate test scenarios that are both aligned with the intended change and effective at revealing unintended bugs, RippleGUItester uses the change intent and code changes of each PR as inputs, as illustrated in Fig 3.

**Change Intent.** For each PR, we collect the PR description as well as any explicitly resolved issue(s), including their summaries and detailed descriptions, if available.

**Code Changes:** Capture the concrete implementation details of a PR.

*Change Description and Modified Files:* Use implementation-level descriptions (e.g., commit messages) to understand how the change is realized in code. Extract fine-grained change information, including modified files, file paths, and detailed patches.

- **Change Intent**: Description of change intent. What is being fixed or implemented?
- **Code Changes**:
  - **Change Description**: Short description of what this code change does.
  - **Files**: List of files changed.
    - **File Path**: The path of the modified file.
    - **File Patch**: The diff or patch showing changes made to this file.
  - **Preceding Change Intents**: A list of prior change intents whose related code is being modified again in this change.

Fig. 3. Input Description in Prompt

*Preceding Change Intents.* As discussed in Section 2.2, code changes may introduce cross-scenario side effects. To mitigate this issue, we leverage historical traceability information in software repositories. For each modified or deleted line in a PR, we use code-commit traceability (e.g., GitHub blame) to identify earlier commits that last modified the same code regions, and retrieve the corresponding PRs and the issues they resolved. We treat the summaries and descriptions of these historically related PRs and issues as preceding change intents, as the changed code regions were originally introduced or modified to satisfy those intents. To prioritize relevance, we rank preceding change intents based on their degree of code overlap with the current PR, with higher overlap indicating a higher likelihood of impact. This design enables RippleGUItester to validate the intended behavior of the current PR while explicitly guarding against regressions affecting previously addressed scenarios. For example, as illustrated in Section 2.2, the PR fixing Issue 1792881 (a scrollbar-related defect) overlaps with code previously modified to support search bar functionality. As a result, the corresponding search-related issues and PRs are identified as preceding change intents, prompting RippleGUItester to generate test scenarios that verify both the scrollbar fix and the preservation of correct search bar behavior.

*3.1.2 Test Scenario Generation.* Generating test scenarios requires analyzing natural-language change intent together with corresponding code changes, which may span multiple programming languages. Traditional change impact analysis techniques [26, 27] are insufficient in this setting, lacking support for multiple languages. Accordingly, we use LLMs to perform test scenario generation, leveraging their capabilities in natural language and multi-language code understanding.

**Initialization.** We initialize the LLM using a system-level role-play prompt tailored for test scenario generation based on change intent and code changes. The prompt consists of three components: *Role Assignment*, which defines the LLM as a test scenario generation tool; *Task Description*, which provides step-by-step instructions; and *Output Guidelines*, which specify the required response format, including fields such as "Change Impact Analysis" and "Test Scenarios".

**Input Description.** As illustrated in Fig 3, the inputs consist of the change intent and its corresponding code changes.

**Impact Analysis and Scenario Generation.** Given the change intent and corresponding code changes, the LLM first analyzes the rationale and desired outcomes of the change and then examines the concrete code modifications that realize this intent. Based on this understanding, it performs change-impact analysis to identify potentially affected end-user scenarios, including high-risk or sensitive cases. Finally, it generates test scenarios derived from the impact analysis, covering both new or modified behaviors and critical existing workflows. All scenarios are written strictly from the end-user perspective, excluding code, internal implementation details, and developer jargon.

For example, as shown in Fig 1, when testing the PR that fixes Issue 1858633, *Test Scenario Generation* identifies that the change affects how the Passwords page is opened from the App Menu and other entry points. Accordingly, RippleGUItester generates test scenarios that exercise this behavior. One such scenario is shown in Fig. 4a, verifying that no fragment is appended when opening Passwords page from an internal `about:` page.

*3.1.3 Event Sequence Enhancement.* After obtaining the initial test scenarios generated by the *Test Scenario Generation*, we further enrich their event sequences to increase event sequence diversity.

- **Explanation:** Ensure no fragment is added when opening Passwords from an internal/about page where no http(s) origin should be preselected.
- **Summary**: Open Passwords from an about: page: no URL fragment
- **Steps**:
  - Open a new tab and navigate to about:newtab (or another about: page).
  - Open the App Menu and click Passwords.

(a) Generated test scenario

- **Explanation:** Exercise a non-browser popup path to verify robust window targeting and fragment-free URL. Page Info-based flows already demonstrate creating a new tab in the browser from a non-tabbed window.
- **Summary**: Trigger from a popup context: opens in main browser window; clean URL; correct preselection
- **Steps**:
  - From Site A, open a popup window (e.g., click a link that opens a separate small window with a login form).
  - In the popup, open Page Info (e.g., Ctrl/Cmd+I) and click View Saved Passwords.

(b) Event-enriched test scenario

- **Test Data:**
  - **Name:** popup_open_script
  - **Requirement:** Simple way to open a small popup window to Site A.
  - **Example:** run in the Web Console on any page: `window.open('https://www.saucedemo.com/','pwmpopup','width=480,height=700')`
- **Summary**: Trigger from a popup context: opens in main browser window; clean URL; correct preselection — Page Info entry point opens the manager in a tab and should target the main window
- **Steps**:
  - From any tab, open a popup to Site A by running: `window.open('https://www.saucedemo.com/','pwmpopup','width=480,height=700')`.
  - In the popup window, open Page Info (Ctrl/Cmd+I) > Security and click View Saved Passwords.

(c) Data-enriched test scenario

Fig. 4. Test Scenario Generation and Enrichment for the PR fixing Issue 1858633.

**Scenario Knowledge Base Construction.** As discussed in Section 2.2, triggering bugs often requires diverse event sequences. Past bug reports provide a rich collection of crowdsourced usage scenarios, making them a valuable knowledge base for event sequence enhancement. For example, past bug reports reveal multiple event sequences for opening the Passwords page, such as through the Preferences page, the autocomplete dropdown footer, the Page Info dialog, save/update doorhangers, which are difficult to enumerate manually. However, not all reports contain meaningful usage scenarios. To address this, we first apply rule-based heuristics to filter out log-oriented reports (e.g., those with excessive timestamps or keywords such as "intermittent"). We then employ LLMs to further identify reports that describe end-user GUI interaction scenarios. These reports form the *Scenario Knowledge Base* (SKB). To enable retrieval, we split these reports into overlapping chunks of fixed token size and embed them into a vector space, supporting both keyword-based and semantic search. During test generation, relevant event sequences are retrieved from the SKB and incorporated via retrieval-augmented generation to enrich event sequence generation.

**Initialization.** The LLM is assigned the role of enhancing test scenarios by introducing diverse event sequences based on the change impact analysis.

**Input Description.** The change intent explanation, change impact analysis, and test scenarios generated by the *Test Scenario Generation* serve as inputs. These inputs guide the retrieval of relevant event sequence knowledge, which is used to enrich the event sequences of the test scenarios.

**Event Sequence Retrieval and Enrichment.** First, the component identifies opportunities for increasing event sequence diversity by analyzing the change impact analysis and the initial test scenarios. Second, it generates targeted queries to retrieve alternative event sequences from historical end-user usage and test scenarios in the SKB. Third, the retrieved event sequences are integrated to enrich the test scenarios, increasing event sequence diversity. Finally, the enriched scenarios are validated to ensure they remain simple, self-contained, and independent. If enrichment introduces excessive complexity, new scenarios are generated instead. All scenarios are written strictly from the end-user perspective, without reference to internal implementation details.

Following the test scenario in Fig. 4a and the change-impact analysis indicating that the PR affects how the Passwords page is opened from different entry points, *Event Sequence Enhancement* queries the SKB to identify additional access event sequences. An enriched scenario is shown in Fig. 4b, where the Passwords page is opened via a popup window and the Page Info interface instead of the App Menu. Notably, opening the Passwords page from a popup window is a critical event for triggering the regression reported in Issue 1937085. At this stage, the scenario still contains

abstract events (e.g., opening a popup window from a website) without concrete test data (e.g., specifying a website that reliably triggers a popup window).

*3.1.4 Test Data Enhancement.* After enriching test scenarios with diverse event sequences, we further identify and instantiate the required test data to make the scenarios executable.

**Initialization.** The LLM is assigned the role of identifying and instantiating the test data required by each scenario.

**Input Description.** The input consists of test scenarios enriched with diverse event sequences.

**Test Data Identification and Enrichment.** For each test scenario, the LLM identifies the required test data by analyzing the scenario and inferring constraints for each data item, and instantiates them into concrete, executable values. Existing examples, if present, are validated against these constraints and replaced if non-compliant. Otherwise, realistic real-world data are generated. To increase coverage, representative data variations are introduced without redundancy, including boundary cases and contrasting valid or invalid inputs when meaningful. Across scenarios, different representative examples are used for the same data type to maximize diversity. The instantiated data are integrated into the scenarios, which are validated to remain simple, self-contained, and independent. If enrichment introduces excessive complexity, new scenarios are generated instead. All scenarios are written strictly from the end-user perspective, without reference to internal implementation details or developer jargon.

Following the scenario in Fig. 4b, *Test Data Enhancement* instantiates abstract data requirements into concrete inputs by synthesizing a script that reliably creates a popup context. As shown in Fig. 4c, this concrete test data enables execution of the scenario that triggers the regression in Issue 1937085, which only manifests when the Passwords page is accessed via a popup window.

### 3.2 Test Scenario Executor

As shown in Fig 2, the Executor executes generated test scenarios on both the pre-change and post-change builds of the SUT. It consists of two components: (1) an LLM-based UI Instruction Translation, which translates high-level scenario steps into structured UI instructions based on the current GUI state; and (2) an algorithmic UI Instruction Execution, which executes these instructions in a containerized environment. This process is repeated until the test scenario is fully executed or a predefined execution budget is reached.

*3.2.1 UI Instruction Translation.* **Initialization.** The *UI Instruction Translation* is initialized with a system-level role-play prompt and maintains a memory session that records previously executed UI instructions. For each scenario step, it translates the high-level description into structured UI instructions and invokes the *UI Instruction Execution* to execute them in a containerized environment. We define eleven standard UI actions: click, right-click, long-click, double-click, triple-click, input, scroll, drag, move, keypress, and wait. Some actions require arguments: click, right-click, long-click, double-click, triple-click, and move require a position; input requires a position and text; scroll requires a direction; and keypress requires key values.

**Input Description.** The inputs consist of test scenarios enriched with diverse event sequences and associated test data, together with the current GUI state of the SUT, represented visually (e.g., as a screenshot) to guide UI instruction translation.

**UI Instruction Translation.** *UI Instruction Translation* analyzes the test scenario together with the current GUI state (e.g., a screenshot) to determine the next step to execute. If the step has unmet preconditions, it generates UI instructions to satisfy them before performing the main step, treating precondition actions as regular steps. It then produces structured UI instructions to carry out the step. Specifically, UI instructions include the action (e.g., click), the name and position of the target GUI element (coordinates), the input text if the action is an input, the keys if the action

is a keypress, and the scroll direction if the action is a scroll. To improve execution efficiency, when multiple UI instructions can be generated without temporal dependencies, they are batched into a single response. For example, when saving a login, if the website, username, and password fields as well as the Save button are available, the component generates instructions to fill the fields and click the Save button in one response.

*3.2.2 UI Instruction Execution.* After structured UI instructions are generated, the *UI Instruction Translation* invokes the *UI Instruction Execution* to execute them on the SUT. The execution component interacts with the GUI using command-line UI utilities (e.g., `xdotool`) to perform actions such as clicking, inputting, and scrolling. After execution, it captures a new screenshot to record the updated GUI state, which is used to guide subsequent steps.

All tests are executed within isolated Docker environments to ensure environmental consistency. For each SUT, we first construct a base Docker image that encapsulates the required build tools, runtime dependencies, and environment configuration. For each PR under test, we derive a Docker image from this base image, within which we build both the pre-change and post-change versions of the SUT to obtain the corresponding executable files. Each test scenario is then executed in a fresh container instance for each build, where the container launches the executable of the corresponding SUT version under test. This setup eliminates interference from residual files, cached state, or configuration artifacts from prior executions. To ensure fair and consistent comparison, the UI instructions generated for the post-change build are reused verbatim for the pre-change build.

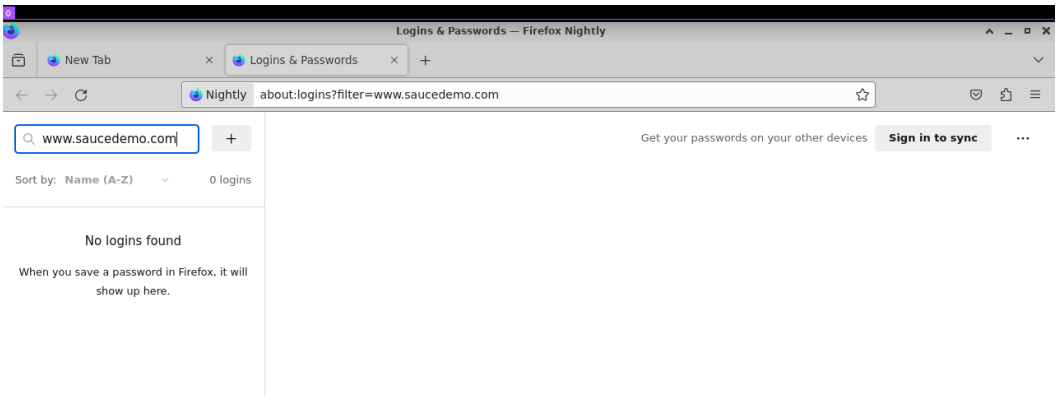
Following the scenario in Fig. 4c, the Executor executes the test on both builds. The Passwords page opens in the main window in the pre-change build (Fig 5a) and in a popup window in the post-change build (Fig 5b), revealing the regression reported in Issue 1937085. Although triggered by different event sequences, both executions expose the same underlying defect.

### 3.3 Bug Detector

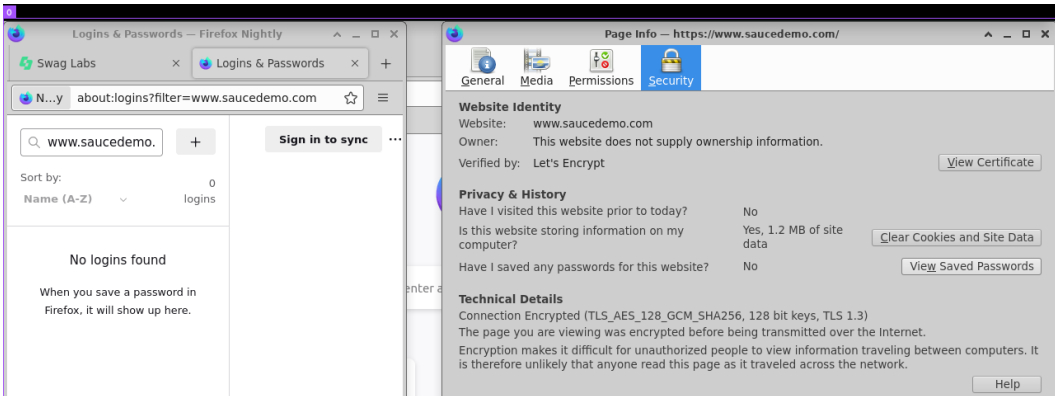
After executing test scenarios on both the pre-change and post-change versions of the software, the Detector analyzes the GUI states to identify unintended behavioral changes. It consists of three components: (1) algorithmic pixel-based GUI difference detection and annotation, (2) LLM-based bug detection, and (3) LLM-based bug filtering.

*3.3.1 Pixel-Based Difference Detection and Annotation.* To automatically identify visual changes between pre-change and post-change GUI states, we first perform pixel-level comparison of the corresponding screenshots to detect differences. Minor variations are filtered using a threshold (set to 30 in this work), after which morphological dilation is applied to connect nearby difference pixels. We then extract connected components to identify contiguous regions of change and generate bounding boxes to localize each differing region. The resulting structured difference information consists of a list of dictionaries, each containing an index and the bounding box coordinates of a detected region, enabling automated identification and localization of visual changes. Finally, these regions are annotated on the screenshots with labeled bounding boxes, providing a clear visualization of the affected GUI elements. Both the structured difference information and the annotated GUI screenshots are used as inputs for subsequent bug detection analysis.

*3.3.2 Bug Detection.* **Initialization.** *Bug Detection* is initialized with a system-level role-play prompt that assigns the LLM the role of interpreting pre-change and post-change GUI differences to identify unintended issues introduced by a code change. The prompt treats GUI differences not explicitly described in the change intent as potential bugs, while excluding transient artifacts such as incomplete renderings and out-of-sync GUI states caused by failed or delayed step execution. A dialogue session is maintained to retain context across detection steps.




(a) Pre-change Build




(b) Post-change Build

Fig. 5. Test Scenario Execution for PR Fixing Issue1858633

**Input Description.** The inputs include the change intent and the corresponding change intent explanation, which details the rationale behind the code changes and is generated by *Test Scenario Generation*. In addition, *Bug Detection* takes the executed test scenario as input. For each UI instruction executed in a test scenario, the following information is provided: (1) the UI instruction executed, (2) a screenshot of the GUI state before the code change, (3) a screenshot of the GUI state after the code change, and (4) *Parsed Info*, a structured representation of detected GUI differences between the two screenshots. Each detected difference is assigned an index and localized by a bounding box (*bbox*: $x_1, y_1, x_2, y_2$), and the corresponding numbered bounding boxes are overlaid on the screenshots to visually highlight the detected differences.

**Bug Detection.** Given the parsed differences and annotated screenshots for each executed action, *Bug Detection* first performs a holistic GUI analysis to summarize the visible interface state and enumerate present components. It then conducts a differential analysis by traversing the *Parsed Info* (indexed differences with *bbox*: $x_1, y_1, x_2, y_2$) together with their visual annotations to describe concrete changes, including element additions or removals, position or size shifts, and content or property updates (e.g., labels and values). Each detected difference is classified as either expected, when it aligns with the change intent and its explanation, or bug, when it constitutes an unintended deviation beyond the specified scope. Beyond per-difference classification, the

*Bug Detection* evaluates the consistency and impact of detected changes on the overall interface, checking for layout misalignment, redundant or missing elements, inconsistent or conflicting states or behaviors, broken or confusing interactions, and potential usability or accessibility degradation. For each unique unintended issue, the *Bug Detection* produces a concise reasoning trace and files a bug report, avoiding speculation and reporting only clear, observable potential defects.

*3.3.3 Bug Filtering.* For each PR under test, RippleGUItester executes multiple test scenarios to maximize coverage. As a result, the same underlying bug may be triggered repeatedly across different scenarios or multiple times within a single scenario. In addition, *Bug Detection* may produce spurious reports due to transient rendering artifacts in screenshots. Consequently, the raw output of *Bug Detection* may contain duplicate or invalid bug reports. To address this issue, we employ an LLM-based post-processing step to filter bug reports before producing the final results.

**Duplicate Bug Filtering.** We identify and remove duplicate bug reports. Two reports are considered duplicates if they describe the same root cause, even if they differ in wording. Among duplicate reports, we retain a single representative instance. **Filtering Rendering-Timing Artifacts.** We filter out false positives caused by screenshot timing or rendering delays. Because screenshots may be captured before the GUI has fully stabilized, transient visual differences, such as missing caret indicators, partially rendered widgets, or temporarily incorrect layout states, may be mistakenly reported as bugs. **Filtering Non-Deterministic GUI Behavior.** We remove bug reports caused by unstable or non-deterministic GUI behavior. These include differences arising from layout shifts, random ordering of UI elements, or inconsistent panel selection across executions. Such behaviors are common in complex GUI applications and do not indicate functional defects.

Following the test scenario execution in Fig. 5, the Detector first applies pixel-based difference detection to identify visual discrepancies between the two GUI states. Because the GUI states differ globally, the entire interface is enclosed within a single purple bounding box (indexed as 0), as shown in Fig. 5. The Detector then analyzes this detected difference in conjunction with the change intent, as illustrated in Fig. 6. Through reasoning over both visual evidence and natural-language intent, it determines that the observed behavioral difference falls outside the intended scope of the change, which targets only URL fragment handling and preselection mechanisms. Accordingly, the Detector generates a bug report describing this unintended behavior.

- **Chain of thoughts:** At this step, both versions open about:logins with a domain filter (no hash) and show the same search prefill. The key visual difference: before-change opens the Passwords tab in the main window; after-change opens it inside the popup window (with the Page Info dialog still on top). This behavior change is attributable to the code change and is outside the change intent, which only targeted URL fragment handling and preselection mechanism.
- **Summary:** Page Info → View Saved Passwords opens about:logins in the popup window instead of the main browser window.
- **Steps:**
  - Open a popup: `window.open('https://www.saucedemo.com/','pwmpopup','width=480,height=700')`
  - In the popup, open Page Info (Ctrl/Cmd+I) → Security
  - Click View Saved Passwords
- **Expected behaviors:**
  - A Passwords (about:logins) tab opens in and focuses the main browser window
  - Address bar should show about:logins (no # fragment) with appropriate preselection
- **Actual behaviors:**
  - After change, the Passwords tab opens inside the popup window, not the main window, and the Page Info dialog remains on top
  - Address bar shows about:logins?filter=www.saucedemo.com (no hash) — preselection appears fine, but window targeting/focus is wrong

Fig. 6. Bug Detection for PR Fixing Issue 1858633

## 4 Experiment

To evaluate the effectiveness and practicality of RippleGUItester in detecting bugs introduced by code changes, we formulate the following research questions:

**RQ1:** How effective is RippleGUItester at detecting previously unknown bugs?

**RQ2:** What is the precision of the bugs detected by RippleGUItester?
**RQ3:** What is the recall of previously known regression bugs?
**RQ4:** What are the computational and monetary costs of RippleGUItester?

### 4.1 Experimental Setup

*4.1.1 Implementation.* Our implementation comprises approximately 13.4K lines of Python code and relies on standard Python libraries for UI automation, image processing, and vector-based retrieval, as well as official SDKs for interfacing with LLM services. To ensure environment consistency and reproducibility, the build of the SUTs and the execution of test scenarios are performed inside Docker containers. Our experiments were conducted on three machines with heterogeneous but comparable hardware configurations: (1) a MacBook Pro equipped with an Apple M4 Pro chip and 24 GB RAM (macOS 26.2), (2) a MacBook Pro with a 2.0 GHz quad-core Intel Core i5 and 16 GB RAM (macOS 14.1.2), and (3) a Linux server with an Intel Core i9-14900 CPU (24 cores, 32 threads, up to 5.8 GHz) and 64 GB RAM, running Ubuntu 24.04.1. We observed consistent results across these machines, suggesting that RippleGUItester is not hardware-dependent.

We employ state-of-the-art LLMs to explore the upper bound of model capability for this task. Specifically, we use GPT-5.2 for the Generator and Detector due to its strong reasoning and multimodal capabilities, and Claude Opus 4.5 for the Executor due to its superior performance in grounding and locating GUI elements. For auxiliary tasks, we use GPT-5 Nano to filter historical bug reports during SKB construction, leveraging its efficiency for lightweight classification, and text-embedding-3-large to encode the SKB into vector representations for retrieval.

*4.1.2 Software Under Test.* We selected four widely used open-source desktop applications: Firefox, a fast and privacy-focused web browser; Zettlr, an all-in-one publication workbench; JabRef, a graphical Java application for managing BibTeX and BibLaTeX databases; and Godot, a multi-platform 2D and 3D game engine. These applications were selected based on diversity, popularity and traceability. First, the projects span a diverse range of application domains, which allows us to assess the generalizability. Second, all projects are highly popular. As of the time of this work, Firefox has over 11k GitHub stars and 6,302 contributors, Zettlr has over 12.4k stars and 151 contributors, JabRef has over 4.2k stars and 874 contributors, and Godot has more than 105k stars with 3,165 contributors. Each project has a long development history of at least eight years and demonstrates sustained active developer engagement, ensuring the representatives of mature and continuously evolving software systems. Third, all projects maintain public issue tracking systems and version control histories, enabling the traceability of code changes and resolved issues.

*4.1.3 Dataset.* For SKB construction, we crawl 246,665 Firefox bugs, from which 74,625 bugs are retained after filtering. For Zettlr, we crawl all issues and pull requests with IDs from 1 to 6,035, retaining 2,832 issues and PRs after filtering. For JabRef, we crawl issues and pull requests with IDs from 1 to 14,554, resulting in 3,256 issues and PRs in the SKB. For Godot, we crawl issues and pull requests with IDs from 1 to 113,618, of which 35,657 issues and PRs remain after filtering. To prevent data leakage, when retrieving SKB entries for a given PR under testing, we only consider issues and PRs created prior to the creation time of that PR.

Firefox provides explicit traceability between code changes and the bugs they introduce, making it suitable for evaluating recall on previously known regression bugs. From Firefox, we randomly select 30 merged PRs, each of which is associated with at least one introduced bug. For Zettlr, Godot, and JabRef, where such fine-grained traceability is not consistently available, we focus on evaluating the ability of RippleGUItester to detect previously unknown bugs under realistic development settings. For Zettlr and Godot, we collect 50 recently merged PRs prior to January 2026. For JabRef, we initially also collected 50 recently merged PRs. However, after filtering out PRs

that are non-functional or unsuitable for testing, the remaining number was insufficient. Therefore, we extend the collection window and retrieve the 80 recently merged PRs of JabRef prior to January 2026. After filtering, we retain 38 PRs for Zettlr, 25 PRs for Godot, and 18 PRs for JabRef.

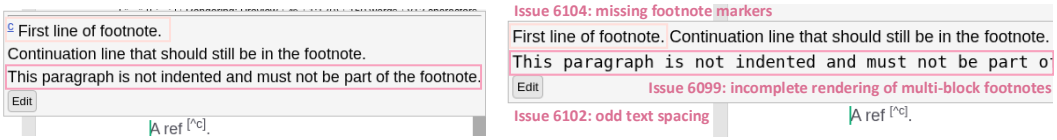
### 4.2 Effectiveness in Detecting Previously Unknown Bugs (RQ1)

We manually inspect all bugs after testing PRs across all SUTs to determine whether RippleGUItester identifies previously unknown bugs. As shown in Table 1, RippleGUItester detected 117 unintended changes. We reported 26 bugs that still existed in the latest project versions to the corresponding development teams, 18 bugs have been fixed, 3 have been confirmed, and 5 were marked as intended. The bug counts in Table 1 reflect deduplicated unknown bugs, whereas the true positive (TP) counts include duplicates. For Firefox, known ground-truth bugs are also excluded for the bug count.

Table 1. Bug Detection

| SUT | PR # | Bug # | TP # | FP # | Prec. |
|---|---|---|---|---|---|
| Firefox | 30 | 29 | 52 | 41 | 0.559 |
| Zettlr | 38 | 63 | 67 | 95 | 0.414 |
| JabRef | 18 | 15 | 17 | 23 | 0.425 |
| Godot | 25 | 10 | 10 | 14 | 0.417 |
| Total | 111 | 117 | 146 | 173 | 0.458 |

We further demonstrate the effectiveness through concrete examples. PR 14489 in JabRef improves the HTTP server code but introduces a bug: HTTP server endpoint/libraries/demo fails, returning HTTP 500 with HTML and breaking demo library retrieval (Issue 14807). After reporting, the bug was promptly fixed. The detection of this bug illustrates RippleGUItester's effectiveness in identifying issues in backend server logic and execution. As another example, PR 5976 in Zettlr refactors the parsing and rendering of Markdown footnotes. RippleGUItester analyzes the potential impact, and generates test scenarios focusing on high-risk behaviors such as multi-block footnotes. After execution, RippleGUItester detects multiple bugs, including incomplete rendering of multi-block footnotes, odd text spacing, incorrect handling of indented continuations, missing footnote markers, and tooltips appearing for invalid footnotes. (Issue 6099, Issue 6102, Issue 6103, Issue 6104, Issue 6106). Fig 7 shows a subset of these bugs. After reporting, all issues were fixed, demonstrating the effectiveness in detecting bugs from non-trivial refactoring changes.




(a) Pre-change Build (b) Post-change Build

Fig. 7. Subset of bugs introduced by PR 5976

We present another example from Godot. PR 113611 fixes missing "+" separators in editor tooltips describing keyboard shortcuts, a change that appears purely cosmetic at first glance. Through change-impact analysis, RippleGUItester flags potential localization risks, with support from preceding change intents (PR 106943 and PR 106946) that altered overlapping code handling tooltip translation. Guided by this analysis, RippleGUItester generates and executes test scenarios for tooltip rendering under language-switching conditions. It detects a bug (Issue 114157 ), where switching the editor to French produces mixed-language shortcut tooltips. While the developers traced this to temporarily outdated translations rather than functional code, RippleGUItester successfully detected the user-visible issue and enabled timely translation updates.

In addition, RippleGUItester detects relatively fewer bugs in Godot. Closer inspection reveals that this is largely due to Godot's testing characteristics. Many Godot bugs require constructing a

minimal reproduction project before the issue can be triggered. Building such projects typically involves non-trivial setup steps, causing test scenarios to frequently exhaust the predefined execution budget before completion. This limitation could be mitigated by increasing the maximum execution budget. Alternatively, execution efficiency could be improved by reusing existing resources. For example, many Godot PRs and issues already include minimal reproduction projects. Incorporating these projects into the scenario knowledge graph or reusing them through other mechanisms could substantially improve the efficiency and effectiveness of test scenario execution for Godot.

### 4.3 Precision of Reported Bugs (RQ2)

We manually label the issues reported by RippleGUItester for all SUTs as TPs or FPs, and compute precision accordingly. As shown in Table 1, RippleGUItester detects 146 TP bugs overall, achieving a precision of 45.8%. On Firefox, RippleGUItester detects 52 TP bugs from 30 PRs (55.9%). On Zettlr, 67 TP bugs are detected from 38 PRs (41.4%). JabRef yields 17 TP bugs from 18 PRs (42.5%), while Godot yields 10 TP bugs from 25 PRs (41.7%). Note that the TP counts include a small number of bugs that predated the evaluated code changes. While RippleGUItester effectively identifies bugs across diverse projects, it also incurs a non-negligible false positive rate.

*4.3.1 GUI Rendering and Temporal Instability (78/173: 45.1%).* This category covers false positives caused by non-deterministic GUI rendering and variability in screenshot timing due to reliance on screenshots to represent GUI states.

**Screenshot timing or rendering delay (39 cases).** The timing of screenshot capture may affect transient visual elements, such as text carets, partially rendered UI components, or delayed dynamic content, leading to benign differences that are mistakenly reported as bugs.

**Unstable GUI layout or rendering (39 cases).** These false positives stem from inherent layout variability, where identical actions produce different visual arrangements across executions. This is common in Zettlr and JabRef. For example, in Zettlr, opening a file may display the document in different split windows. In JabRef, the relative ordering of sidebar components, such as the Web Search and Groups panels, may vary across executions.

A potential mitigation is to execute the same UI instructions multiple times and check for consistent GUI states, filtering out transient and non-deterministic visual differences.

*4.3.2 LLM Reasoning and Hallucination Errors (49/173: 28.3%).* This category includes false positives caused by limitations in the LLM's reasoning and grounding.

**Misinterpreted or hallucinated GUI rendering (40 cases).** Some false positives arise from how pixel-level differences are annotated in screenshots during detection. Annotated regions may correspond to expected GUI changes, but the LLM can misinterpret them as defects. For example, in one case, a code change intentionally relocates the "Apply" button within the "Clean up entries" dialog. Although the GUI is correctly updated after the change, the detection annotates the original button location as a differing region, rendering an empty placeholder box in the post-change screenshot. The LLM then interprets this annotated region as a missing or blank button, incorrectly reporting it as a bug. Other cases involve pure hallucination, where the LLM reports non-existent GUI elements (e.g., an extra edit icon) that do not appear in either screenshot.

**Assumed execution of unperformed actions (6 cases).** In these cases, the LLM incorrectly infers GUI states based on actions that were not executed. For example, it reports a missing focus indicator on an input field even though the focus action never occurred.

**Unnecessary improvement suggestions (3 cases).** Here, the LLM flags intended behavior as a bug. For instance, it reports a disabled "Rename" button in Zettlr as an issue, despite this being expected when no renaming action is initiated.

False positives can be mitigated by clarifying the semantics of visual annotations through prompt engineering, preventing the LLM from treating detection markers as GUI elements. For hallucinations caused by incorrect action inference, model fine-tuning with execution-aware data could improve consistency between execution traces and visual reasoning.

*4.3.3 GUI Interaction and Replay Instability (26/173: 15.0%).* This category captures false positives caused by non-deterministic GUI interactions and inconsistencies during interaction replay.

**GUI element localization or interaction failure (15 cases).** These false positives arise when UI elements are incorrectly located or intended interactions are not executed as planned. As a result, the executed interactions deviate from the test scenario, producing GUI states that are mistakenly reported as bugs. For example, the LLM reports that pressing Ctrl+A followed by Backspace fails to clear the address field, but inspection shows that the field was never correctly focused. In another case, resizing a dialog by dragging fails due to a limitation in our execution implementation, which only specifies the drag start point and relies on a default end point.

**GUI differences causing replay mismatch (11 cases).** Here, legitimate GUI changes introduced by the code cause replayed interactions to become misaligned. UI coordinates that are valid in the post-change build may not correspond to the correct elements in the pre-change build, leading to inconsistent behavior. For example, a code change adjusts the position of the password input field. During execution on the post-change build, the UI interaction correctly clicks the relocated password field. However, when the same interaction is replayed on the pre-change build, it fails to target the corresponding field due to the layout difference. As a result, a popup is triggered in the post-change execution, while no such popup appears in the pre-change execution.

False positives can be mitigated by improving the robustness of UI execution (e.g., specifying full drag trajectories), enhancing element localization via execution-aware LLM fine-tuning, and integrating LLMs into the replay phase to detect and adapt to interaction failures.

*4.3.4 Expected GUI Differences due to Code Changes (18/173: 10.4%).* The detected visual differences correctly reflect modified behavior or UI states introduced by the PR, but are nonetheless flagged as bugs. For example, the "Commit" button is shown in a disabled (greyed-out) state in the post-change build, whereas it is enabled in the pre-change build. This difference is an intended result of updated validation logic, which disables the button until specific conditions are met.

False positives can be mitigated through prompt engineering. The detection prompt can more explicitly emphasize the need to distinguish between unintended bugs and legitimate GUI changes.

*Others (2/173: 1.2%).* This small category includes a replay instability issue. The test scenario execution generates a PDF file with randomized file names. During replay, the execution attempts to reuse the file name generated in the play phase to ensure consistency. However, the corresponding file does not exist because the replay generates a different file name. This mismatch leads to file access failures during replay and results in a false positive.

### 4.4 Recall on Previously Known Regression Bugs (RQ3)

We focus exclusively on Firefox, leveraging its known ground-truth bugs to assess the extent to which RippleGUItester can discover known bugs. As shown in Table 1, after testing 30 PRs, RippleGUItester detected 52 TP bugs, among which 10 were duplicates. After deduplication and excluding one bug that was present both before and after the code change, 41 unique TP bugs remained. We further compare the detected bugs with the ground-truth bugs documented in Firefox's issue tracking system. Among the 30 PRs, each introduced at least one bug according to the issue tracking records, yielding a total of 54 ground-truth bugs. Notably, 6 of ground-truth bugs are primarily described through logs or stack traces (e.g., Issue 1923374), making it difficult to precisely infer the corresponding buggy GUI behavior. As a result, during evaluation, we could not

reliably determine whether the bugs detected by RippleGUItester correspond to these log-described ground-truth bugs. Therefore, we excluded these bugs. In addition, two bugs marked as duplicates were excluded. The final ground-truth set used for evaluation contains 46 bugs. As illustrated in Fig 8, 13 unique detected bugs by RippleGUItester overlap with the ground-truth bugs, while 33 ground-truth bugs were not detected by RippleGUItester.

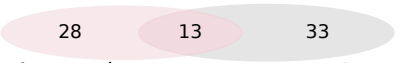



Fig. 8. Bug Distribution.
*Left:* Unique TP bugs by RippleGUItester.
*Right:* Unique Ground-Truth bugs.

*Analysis of Missed Ground-Truth Bugs.* We identified two categories of causes: limitations in test scenario generation coverage and incomplete test scenario execution.

**Limitations in test scenario generation coverage (20/33: 60.6%).** The generated test scenarios lack coverage of the critical trigger steps required to reveal the ground-truth bugs. These bugs often involve hard-to-predict or strict trigger steps that are difficult to anticipate during test scenario generation.

*Hard-to-predict trigger steps (15/20).* The PR fixing Issue 1745734 (Insecure connection icon is barely visible on HTTP website login forms with the dark theme) introduced Issue 1935402. Specifically, clicking and dragging the "This connection is not secure" panel causes its contents to become invisible. Such trigger steps involve uncommon interactions (click and drag a panel) and are thus hard for test scenarios to cover. As a potential improvement, randomization can be introduced into test scenario generation, such as injecting random interaction actions, to increase the unpredictability and diversity of generated test scenarios and thereby improve coverage.

*Strict trigger steps (5/15).* The PR fixing Issue 1592682 introduced Issue 1725969, where the "Create New Login" mode is not dismissed after clicking the "Save" button when the "Username" sort option is active. The generated test scenarios exercised the sorting functionality and the create-new-login functionality under various preconditions, but did not cover the specific combination where a new login is created while the username sort is active, thereby missing this bug. In this work, to control the cost of experiments, we impose an upper bound of seven generated test scenarios per PR. Relaxing this limit and allowing more exhaustive scenario generation could potentially mitigate such missed cases.

**Incomplete test scenario execution (13/33: 39.4%).** Although the test scenarios contain the required trigger steps, the bugs are missed due to incomplete or incorrect execution.

*Reaching the maximum execution budget (4/13).* The introduced Issue 1934682 requires long preconditions to be satisfied. As a result, the execution of the generated test scenario reaches the predefined maximum execution budget before the trigger steps are exercised, causing the bug to be missed. Specifically, we impose two execution limits: (1) a maximum LLM interaction budget, which bounds the number of LLM invocations during scenario execution, and (2) a maximum UI instruction budget, which limits the total number of UI interactions executed. In our experiments, these limits are set to 20 LLM interactions and 35 UI instructions, respectively. As a potential improvement, increasing the execution budget could help alleviate this issue.

*Incorrect execution (9/13).* This category consists of five execution-related failure modes that prevent test scenarios from being correctly completed. (i) *Trigger step execution errors* (4 cases), where interactions required in specific UI components (e.g., password sidebar) are incorrectly executed in other UI contexts (e.g., password page), preventing trigger steps from being properly exercised; (ii) *execution element location errors* (2 cases), where the execution component interacts with incorrect screen coordinates (e.g., clicking the "Create New Login" button at an incorrect location), causing execution to become stuck; (iii) *build failures before or after code changes* (1 case), which cause some test scenarios to remain unexecuted; (iv) *incorrect UI interaction implementation*

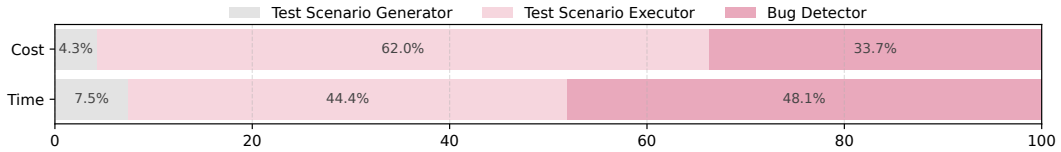



Fig. 9. Overhead breakdown: Averages are $6.00 and 54.8 mins per PR.

(1 case), where certain UI interactions (e.g., drag action) require multiple coordinate pairs to specify the drag trajectory, but the current implementation provides only a single coordinate pair, leading to execution failure; and (v) *Replay failures* (1 case) occur when test scenarios generate randomly named files during execution. During replay, the same UI instructions attempt to access these files, which no longer exist due to the random naming. This causes replay failures.

As potential improvements, constructing a more comprehensive scenario knowledge graph that models user tasks with fine-grained interactive steps could better guide test execution. For other execution-related issues, such as build failures, incorrect UI interaction implementation, and replay failures, more robust engineering solutions could be adopted, e.g., by incorporating LLMs into the replay process to enable adaptive control and error recovery.

### 4.5 Computational and Monetary Costs (RQ4)

SKB construction is a one-off activity. For Zettlr, it takes 3.22 hours and costs $0.50. For JabRef, it takes 5.12 hours and costs $0.73. For Godot, it takes 46.21 hours and costs $5.43. For Firefox, it takes 50.28 hours and costs $20.82. Testing each PR incurs an average time of 54.8 mins and a cost of $6.00 across all SUTs. As shown in Fig 9, Generator incurs relatively low time and cost overhead, accounting for 7.5% of the total execution time and 4.3% of the total cost, with an average of 4.10 mins and $0.26 per PR. Executor constitutes the dominant source of overhead, accounting for 44.4% of the total time and 62.0% of the total cost, with an average of 24.3 mins and $3.72 per PR. Detector accounts for 48.1% of the total execution time and 33.7% of the total cost, with an average of 26.4 mins and $2.02 per PR.

The relatively high overhead is primarily attributed to Executor and Detector. Both stages require processing image-based inputs to reason about the current GUI state, generate corresponding UI interaction instructions, and determine whether observed behavioral differences are intended or unintended. Since our evaluation targets desktop applications, the GUI screenshots are typically of high resolution, resulting in a larger number of image tokens being processed by the model and thus higher computational cost. In our implementation, we employ state-of-the-art MLLMs (i.e., GPT-5.2 and Claude Opus 4.5) to evaluate the approach under strong model capabilities. This choice allows us to assess the effectiveness of RippleGUItester without confounding factors introduced by weaker models, and provides an upper bound on achievable performance under current LLMs.

Although the time and cost per PR may appear relatively high, it is important to note that RippleGUItester analyzes code changes to identify system-level bugs introduced by the corresponding PRs. Performing such analysis manually would still require non-trivial developer effort and incur considerable time and labor costs. Moreover, RippleGUItester complements existing manual testing and code review practices. Even in mature and rigorously reviewed projects such as Firefox, RippleGUItester is still able to identify bugs that are missed by existing testing and review processes. By identifying bugs at the PR stage, RippleGUItester can prevent defective code from being merged, thereby improving overall software quality and user satisfaction. Furthermore, since the detected bugs are directly linked to specific code changes, the cost of fixing them is reduced, as developers can resolve the issues without extensive debugging or localization efforts.

## 5 Threats to Validity

The manual labeling of detected bugs as TPs or FPs may introduce subjective bias. To mitigate this threat, the annotator reviewed all detected bugs twice. A related threat concerns the underlying assumption behind RippleGUItester that observed GUI differences not aligned with the stated change intent indicate unintended bugs. While this assumption enables RippleGUItester to systematically detect and report potential bugs, the stated change intent may not fully specify all acceptable behavioral outcomes of a code change. In particular, some behavioral differences may correspond to undocumented intended behaviors, beneficial but initially unanticipated changes, or necessary consequential changes arising from implementing the intended modification. Consequently, some reported bugs may ultimately be regarded by developers as intended behavior, which may also influence the reported true positives. Future work will systematically investigate these categories and develop more comprehensive methods for distinguishing genuine software defects from acceptable behavioral consequences of code changes. Another threat is the presence of false positives in the bugs reported by RippleGUItester. To mitigate this threat, we further conducted an in-depth analysis of the main sources of false positives and discussed concrete improvement directions.

Our evaluation was conducted on four open-source GUI-based desktop applications, which may not represent all types of software systems. We mitigate this threat by selecting systems from different domains (web browser, document editor, reference manager, and game engine) with large and actively maintained codebases. Furthermore, RippleGUItester is not tailored to any specific application or framework, suggesting that the approach can be adapted to other GUI-based systems. Another threat concerns the choice of LLMs and computational resources. We used state-of-the-art models to explore the upper bound of achievable performance, and results may differ when using smaller or less capable models. However, this design choice allows us to evaluate the feasibility and potential of change-aware GUI testing without conflating methodological limitations with model constraints. Future work can explore cost–performance trade-offs with alternative models. Execution time could also be further reduced through engineering optimizations, such as parallelizing the execution of generated tests.

## 6 Related Work

*Exploratory Testing.* Exploratory testing has been shown to be effective at the system testing level [1, 3, 4, 9, 16, 19, 23, 36, 39]. A variety of principles and guidelines for conducting and managing exploratory testing have been proposed [3, 7, 8, 23, 38]. Several tools and techniques have been developed to support exploratory testing. Tapir [6] generates navigational test cases to reduce duplication. Scenario-based exploratory testing, also known as soap opera testing [7], designs complex and realistic usage scenarios to provoke failures that are difficult to anticipate upfront. Recent work [30–32] supports soap opera testing by constructing a system knowledge graph that captures user tasks and failures, and by generating test scenarios through combinations of related behaviors. SoapOperaTG [31] is a tool for generating and presenting such test scenarios. Su et al. [34] automate the execution of these exploratory test scenarios and associated bug detection with human-like creativity and intelligence. Despite their strong capabilities in test scenario generation, execution, and bug detection, these approaches are not code-change aware, as they do not leverage information about the specific code changes being introduced. In contrast, our work is explicitly triggered by a code change and systematically explores potentially impacted scenarios to uncover bugs introduced by that change.

*Change Impact Analysis.* Change Impact Analysis has been widely studied to reason about the effects of code modifications [2, 10, 21, 26, 27, 41]. Ryder and Tip [27] characterize change impact in object-oriented programs and highlight how language features such as inheritance and dynamic

dispatch lead to non-local effects. Ren et al. propose Chianti [26], a practical CIA tool for Java that identifies affected regression tests by analyzing code-level dependencies between program versions. These approaches are effective at the program and test level but are largely limited to single programming languages and rely on static or dynamic code analysis. They do not reason about end-user GUI behavior. In contrast, our work is change-driven but language-agnostic, and explores the ripple effects of code changes at the level of user-visible GUI interactions.

*Change-aware Regression Testing.* Our work is also related to regression test selection and prioritization [11, 14, 42], but fundamentally differs in that we generate new tests tailored to the given code change. Recent work has proposed techniques that leverage code changes or pull requests to guide regression testing and validation [13, 25, 44]. ChaCo [44] focuses on last-mile regression test augmentation by identifying code changes in a pull request and selecting or augmenting existing tests to improve coverage of the modified code. The approach operates at the code and test level and aims to improve regression test adequacy for specific changes, but it does not reason about GUI-level effects induced by a change. ChangeGuard [13] validates code changes via learning-guided execution, using pairwise comparisons between program states before and after a change. While effective at detecting behavioral differences, the approach primarily targets program-level executions and does not explicitly model end-user interaction scenarios or GUI behaviors. Testora [25] leverages natural language information associated with pull requests to classify behavioral differences exposed by generated tests as intended or unintended. Testora focuses on API-level regression detection in single-language codebases and does not consider GUI-level interactions. In contrast to these approaches, our work targets GUI-centric software systems and explicitly treats a code change as the epicenter of ripple effects across multiple user-visible scenarios, enabling the discovery of indirect and cross-scenario bugs that are difficult to anticipate at the code or API level.

*Scenario Knowledge Mining.* Prior work has explored mining execution information from bug reports or issues to reproduce bugs, such as translating steps-to-reproduce into test cases [12, 28, 43], or reconstructing execution traces from bug videos [5, 15]. Some studies extract structured information from bug reports to support exploratory testing [7, 30, 31, 34]. BUGINE [22, 35] recommends related bugs across applications to assist developers during testing. While these approaches demonstrate the value of crowdsourced artifacts, they do not leverage scenario knowledge to enrich event sequences for testing code changes.

## 7 Conclusions

Code changes can introduce bugs despite extensive testing and code review. Our analysis of real-world software systems shows that a non-trivial fraction of bugs still escape existing tests due to complex event sequences, challenging test data requirements, cross-scenario side effects, and hard-to-predict test oracles. We present RippleGUItester, a change-driven exploratory GUI testing system that (i) generates and enriches test scenarios targeted at code changes through LLM-based change-impact analysis and retrieval-augmented generation over a Scenario Knowledge Base, (ii) executes these scenarios on both pre-change and post-change builds, and (iii) detects unintended behavioral differences through multi-modal differential analysis. Our evaluation on real-world software systems demonstrates that RippleGUItester is effective in practice, uncovering previously unknown bugs that persist in the latest versions. These results highlight the practical value of change-aware exploratory GUI testing in real-world development workflows. We further analyze observed limitations, including major sources of false positives and missed bugs, and outline concrete mitigation strategies and directions for future improvements. More broadly, by explicitly linking code changes to observable user behavior, RippleGUItester lays the groundwork for testing interactive systems with increasingly dynamic and evolving user interfaces.

## Data Availability

Replication package with code and dataset [33]: https://github.com/SuYanqi/RippleGUItester.

## Acknowledgments

This work was partially supported by the German Research Foundation (DFG; projects 492507603, 516334526, and 526259073). Chunyang was partially supported by an Amazon research gift fund.

## References

[1] Wasif Afzal, Ahmad Nauman Ghazi, Juha Itkonen, Richard Torkar, Anneliese Andrews, and Khurram Bhatti. 2015. An experiment on the effectiveness and efficiency of exploratory testing. *Empirical Software Engineering* 20 (2015), 844–878. doi:10.1007/s10664-014-9301-4

[2] Robert S Arnold. 1996. *Software change impact analysis*. IEEE Computer Society Press.

[3] Jonathan Bach. 2000. Session-based test management. *Software Testing and Quality Engineering Magazine* 2, 6 (2000), 32–37.

[4] James Bach. 2004. Exploratory testing. *The testing practitioner* (2004), 253–265.

[5] Carlos Bernal-Cárdenas, Nathan Cooper, Kevin Moran, Oscar Chaparro, Andrian Marcus, and Denys Poshyvanyk. 2020. Translating video recordings of mobile app usages into replayable scenarios. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 309–321. doi:10.1145/3377811.3380328

[6] Miroslav Bures, Karel Frajtak, and Bestoun S Ahmed. 2018. Tapir: Automation support of exploratory testing using model reconstruction of the system under test. *IEEE Transactions on Reliability* 67, 2 (2018), 557–580. doi:10.1109/TR.2018.2799957

[7] Hans Buwalda. 2004. Soap opera testing. *Better Software* 6, 2 (2004), 30–37.

[8] JD Cem Kaner. 2013. An introduction to scenario testing. *Florida Institute of Technology, Melbourne* (2013), 1–13.

[9] JD Cem Kaner and James Bach. 2006. The nature of exploratory testing. (2006).

[10] Andrea De Lucia, Fausto Fasano, and Rocco Oliveto. 2008. Traceability Management for Impact Analysis. In *2008 Frontiers of Software Maintenance*. IEEE, 21–30. doi:10.1109/FOSM.2008.4659245

[11] Sebastian Elbaum, Alexey G Malishevsky, and Gregg Rothermel. 2002. Test case prioritization: A family of empirical studies. *IEEE transactions on software engineering* 28, 2 (2002), 159–182. doi:10.1109/32.988497

[12] Mattia Fazzini, Martin Prammer, Marcelo d'Amorim, and Alessandro Orso. 2018. Automatically translating bug reports into test cases for mobile apps. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 141–152. doi:10.1145/3213846.3213869

[13] Lars Gröninger, Beatriz Souza, and Michael Pradel. 2025. ChangeGuard: Validating code changes via pairwise learning-guided execution. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 936–956. doi:10.1145/3715760

[14] Mary Jean Harrold, David Rosenblum, Gregg Rothermel, and Elaine Weyuker. 2002. Empirical studies of a prediction model for regression test selection. *IEEE Transactions on software engineering* 27, 3 (2002), 248–263. doi:10.1109/32.910860

[15] Madeleine Havranek, Carlos Bernal-Cárdenas, Nathan Cooper, Oscar Chaparro, Denys Poshyvnayk, and Kevin Moran. 2021. V2S: a tool for translating video recordings of mobile app usages into replayable scenarios. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 65–68. doi:10.1109/ICSE-Companion52605.2021.00037

[16] Juha Itkonen, Mika V Mäntylä, and Casper Lassenius. 2012. The role of the tester's knowledge in exploratory software testing. *IEEE Transactions on Software Engineering* 39, 5 (2012), 707–724. doi:10.1109/TSE.2012.55

[17] Yasutaka Kamei, Emad Shihab, Bram Adams, Ahmed E Hassan, Audris Mockus, Anand Sinha, and Naoyasu Ubayashi. 2012. A large-scale empirical study of just-in-time quality assurance. *IEEE Transactions on Software Engineering* 39, 6 (2012), 757–773. doi:10.1109/TSE.2012.70

[18] Cem Kaner. 2008. A tutorial in exploratory testing. *Tutorial presented at QUEST2008.(Available online at: http://www.kaner. com/pdfs/QAIExploring. pdf, accessed: 26 Jan 2014)* (2008).

[19] Cem Kaner, Jack Falk, and Hung Q Nguyen. 1999. *Testing computer software*. John Wiley & Sons.

[20] Sunghun Kim, E James Whitehead, and Yi Zhang. 2008. Classifying software changes: Clean or buggy? *IEEE Transactions on software engineering* 34, 2 (2008), 181–196. doi:10.1109/TSE.2007.70773

[21] James Law and Gregg Rothermel. 2003. Whole program path-based dynamic impact analysis. In *25th International Conference on Software Engineering, 2003. Proceedings.* IEEE, 308–318. doi:10.1109/ICSE.2003.1201210

[22] Ziqiang Li and Shin Hwei Tan. 2020. Bugine: a bug report recommendation system for Android apps. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Companion Proceedings*. 278–279. doi:10.1145/3377812.3390906

[23] James Lyndsay and Neil Van Eeden. 2003. Adventures in session-based testing. *Workroom Productions Ltd. May* 27 (2003).

[24] Shaikh Mostafa, Rodney Rodriguez, and Xiaoyin Wang. 2017. Experience paper: a study on behavioral backward incompatibilities of Java software libraries. In *Proceedings of the 26th ACM SIGSOFT international symposium on software testing and analysis*. 215–225. doi:10.1145/3092703.3092721

[25] Michael Pradel. 2026. Testora: Using Natural Language Intent to Detect Behavioral Regressions. In *International Conference on Software Engineering (ICSE)*.

[26] Xiaoxia Ren, Fenil Shah, Frank Tip, Barbara G Ryder, and Ophelia Chesley. 2004. Chianti: a tool for change impact analysis of java programs. In *Proceedings of the 19th annual ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. 432–448. doi:10.1145/1035292.1029012

[27] Barbara G Ryder and Frank Tip. 2001. Change impact analysis for object-oriented programs. In *Proceedings of the 2001 ACM SIGPLAN-SIGSOFT workshop on Program analysis for software tools and engineering*. 46–53. doi:10.1145/379605.379661

[28] Feng Sidong and Chen Chunyang. 2024. Prompting Is All Your Need: Automated Android Bug Replay with Large Language Models. In *Proceedings of the 46th International Conference on Software Engineering (ICSE'24)*. doi:10.1145/3597503.3608137

[29] Jacek Śliwerski, Thomas Zimmermann, and Andreas Zeller. 2005. When do changes induce fixes? *ACM sigsoft software engineering notes* 30, 4 (2005), 1–5. doi:10.1145/1082983.1083147

[30] Yanqi Su, Zheming Han, Zhenchang Xing, Xin Xia, Xiwei Xu, Liming Zhu, and Qinghua Lu. 2022. Constructing a System Knowledge Graph of User Tasks and Failures from Bug Reports to Support Soap Opera Testing. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13. doi:10.1145/3551349.3556967

[31] Yanqi Su, Zheming Han, Zhenchang Xing, Xiwei Xu, Liming Zhu, and Qinghua Lu. 2023. SoapOperaTG: A Tool for System Knowledge Graph Based Soap Opera Test Generation. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 51–54. doi:10.1109/ICSE-Companion58688.2023.00023

[32] Yanqi Su, Dianshu Liao, Zhenchang Xing, Qing Huang, Mulong Xie, Qinghua Lu, and Xiwei Xu. 2024. Enhancing exploratory testing by large language model and knowledge graph. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12. doi:10.1145/3597503.3639157

[33] Yanqi Su, Michael Pradel, and Chunyang Chen. 2026. RippleGUItester: Replication Package. doi:10.5281/zenodo.21671184

[34] Yanqi Su, Zhenchang Xing, Chong Wang, Chunyang Chen, Sherry Xu, Qinghua Lu, and Liming Zhu. 2025. Automated Soap Opera Testing Directed by LLMs and Scenario Knowledge: Feasibility, Challenges, and Road Ahead. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 757–778. doi:10.1145/3715752

[35] Shin Hwei Tan and Ziqiang Li. 2020. Collaborative bug finding for android apps. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 1335–1347. doi:10.1145/3377811.3380349

[36] Jarle Våga and Ståle Amland. 2002. Managing high-speed web testing. In *Software quality and software testing in internet times*. Springer, 23–30.

[37] Ming Wen, Rongxin Wu, Yepang Liu, Yongqiang Tian, Xuan Xie, Shing-Chi Cheung, and Zhendong Su. 2019. Exploring and exploiting the correlations between bug-inducing and bug-fixing commits. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 326–337. doi:10.1145/3338906.3338962

[38] James A Whittaker. 2009. *Exploratory software testing: tips, tricks, tours, and techniques to guide test design*. Pearson Education.

[39] Bill Wood and David James. 2003. Applying session-based testing to medical software. *Medical Device and Diagnostic Industry* 25, 5 (2003), 90–103.

[40] Zuoning Yin, Ding Yuan, Yuanyuan Zhou, Shankar Pasupathy, and Lakshmi Bairavasundaram. 2011. How do fixes become bugs?. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*. 26–36. doi:10.1145/2025113.2025120

[41] Annie Tsui Tsui Ying, Gail C Murphy, Raymond Ng, and Mark C Chu-Carroll. 2004. Predicting source code changes by mining change history. *IEEE transactions on Software Engineering* 30, 9 (2004), 574–586. doi:10.1109/TSE.2004.52

[42] Shin Yoo and Mark Harman. 2012. Regression testing minimization, selection and prioritization: a survey. *Software testing, verification and reliability* 22, 2 (2012), 67–120. doi:10.1002/stvr.430

[43] Yu Zhao, Tingting Yu, Ting Su, Yang Liu, Wei Zheng, Jingzhi Zhang, and William GJ Halfond. 2019. Recdroid: automatically reproducing android application crashes from bug reports. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 128–139. doi:10.1109/ICSE.2019.00030

[44] Zitong Zhou, Matteo Paltenghi, Miryung Kim, and Michael Pradel. 2026. Change And Cover: Last-Mile, Pull Request-Based Regression Test Augmentation. *arXiv preprint arXiv:2601.10942* (2026).

Received 2026-01-30; accepted 2026-04-16